Nihat Hasan
CMPEN/EE 454
Project 2: Point and Inverse Camera Projection
July 28, 2026

**Introduction**

In my opinion, the goal of this project was to implement the full camera projection pipeline in MATLAB. The motion capture dataset provided the 3D coordinates of twelve body joints across a sequence of frames, while the calibration files contained the intrinsic and extrinsic parameters for two cameras, Vue2 and Vue4. I first projected the original 3D joints into 2D pixel coordinates for both camera views. I then converted those projected pixels back into 3D viewing rays and used pairs of rays from the two cameras to reconstruct the original joint locations via triangulation.

After reconstructing the joints, I measured the Euclidean distance between each original and recovered 3D point. I evaluated the reconstruction for every frame in which all twelve joints had valid confidence values. I also visualized the projected skeletons on the video frames, compared the original and reconstructed 3D skeletons, generated epipolar lines between the two views, and used the MATLAB profiler to examine the program's runtime.

**Procedural Approach & Program Structure**

The program begins by loading the motion capture dataset and the calibration structures for Vue2 and Vue4. The motion capture array contains the 3D coordinates and confidence values for each of the twelve joints. Frames were considered valid only when all twelve confidence values were 1.

For each valid frame, the twelve 3D joint coordinates were extracted and sent to 'myProjectPoints'. This function converted the points to homogeneous coordinates and applied the camera matrices using x-tilde = KPX, where K is the intrinsic matrix, and P is the extrinsic matrix. The first two homogeneous image coordinates were then divided by the third to obtain ordinary pixel coordinates.

The projected pixels from each camera were passed to 'myPixelToRay'. This function removed the intrinsic transformation using the inverse of K, rotated the resulting directions from camera coordinates into world coordinates using R^T, and normalized each direction to unit length. Each viewing ray was represented by the camera position and its unit direction vector.

Corresponding rays from Vue2 and Vue4 were then passed to 'myTriangulateRays'. The function calculated the closest point on each ray and used the midpoint between those two points as the reconstructed 3D joint location. Finally, the Euclidean distance between each original and reconstructed joint was calculated and stored for quantitative evaluation.

The main script also called separate functions to draw the 2D and 3D skeletons, display the minimum- and maximum-error frames, generate the epipolar-line visualization, and evaluate the reconstruction over the full valid frame sequence.

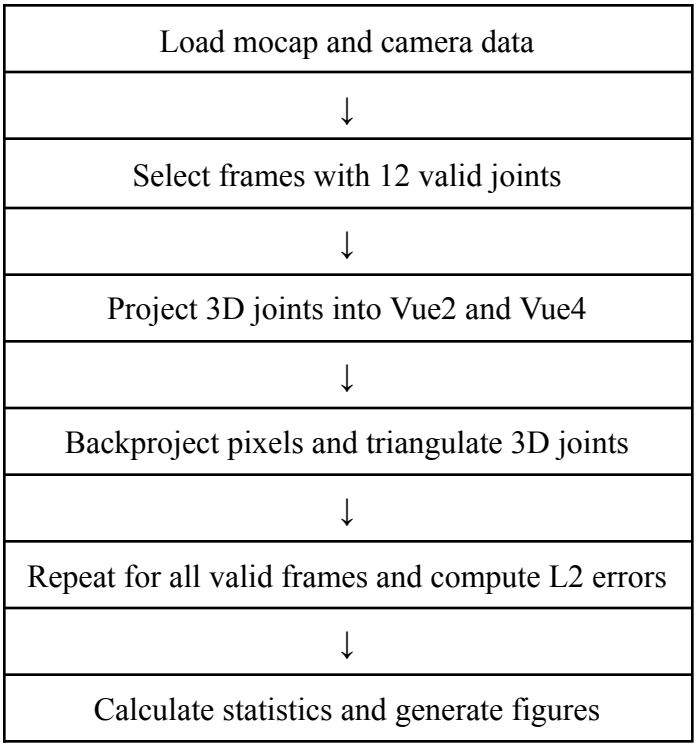


Figure 1. Program flowchart.

**Camera Parameters**

Each camera calibration structure contains intrinsic, extrinsic, and lens-distortion parameters. The intrinsic parameters describe how camera coordinates are converted to image pixel coordinates. These include the focal length, principal point, aspect ratio, and skew. They are combined in the 3x3 matrix 'Kmat'. The 'radial' field contains nonlinear lens-distortion coefficients, but these were ignored as instructed in the project description.

The extrinsic parameters describe the camera's location and orientation relative to the world coordinate system. The 'position' field gives the camera center C in world coordinates. The Vue2 camera center was (-4450.06, 5557.92, 1949.06), and the Vue4 camera center was (4423.56, 5490.36, 1889.02). The orientation is provided as a unit quaternion and as the rotation matrix 'Rmat'. The rotation matrix converts world-coordinate directions into camera coordinates.

The provided extrinsic matrix Pmat follows P = [R | -RC], where R is 'Rmat' and C is the camera position written as a column vector. I verified this relationship in MATLAB by constructing the matrix from 'Rmat' and 'position' and calculating the Frobenius norm of its difference from the provided 'Pmat'. The difference was exactly 0 for both Vue2 and Vue4, confirming that the camera position and rotation were interpreted correctly.

The complete forward projection was performed using x-tilde = KPX, where X is a homogeneous 3D world point, and x-tilde is the resulting homogeneous image point. The ordinary pixel coordinates were obtained by dividing the first and second entries of x-tilde by its third entry.

| Parameter | Meaning | Type |
|---|---|---|
| foclen | Camera focal length | Intrinsic |
| prinpoint | Principal point in pixels | Intrinsic |
| aspectratio | Pixel aspect ratio | Intrinsic |
| skew | Image-axis skew | Intrinsic |
| radial | Radial distortion coefficients | Intrinsic, ignored |
| position | Camera center in world coordinates | Extrinsic |
| orientation/Rmat | Camera orientation | Extrinsic |
| Kmat | Intrinsic calibration matrix | Intrinsic |
| Pmat | Matrix [R \| -RC] | Extrinsic |

Table 1. Camera-calibration fields used in the projection pipeline.

**Qualitative Results**

The first visual check involved projecting the 3D mocap joints into the Vue2 and Vue4 video frames. The projected points were connected into a skeleton using the shoulder, elbow, wrist, hip, knee, and ankle joints. The points followed the person's body shape in both views, confirming that the forward-projection function was working correctly. Small offsets were expected because the nonlinear radial-distortion parameters were ignored.

The inverse-projection and triangulation results were verified by plotting the original and reconstructed 3D skeletons on the same axes. The two skeletons overlapped almost exactly, consistent with the very small numerical reconstruction errors produced by the program.

I also displayed the frames with the minimum and maximum total reconstruction error. The minimum-error frame was 23743, while the maximum-error frame was 16616. For both frames, the projected 2D skeletons still matched the person in the Vue2 and Vue4 images, and the original and reconstructed 3D skeletons remained almost indistinguishable.

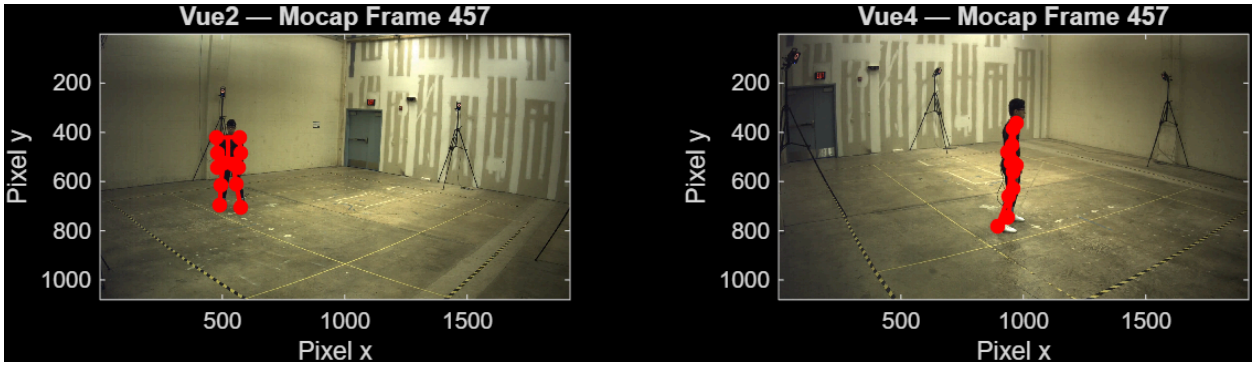



Figure 2. Projected 2D skeletons overlaid on Vue2 and Vue4 video frames for mocap frame 457.

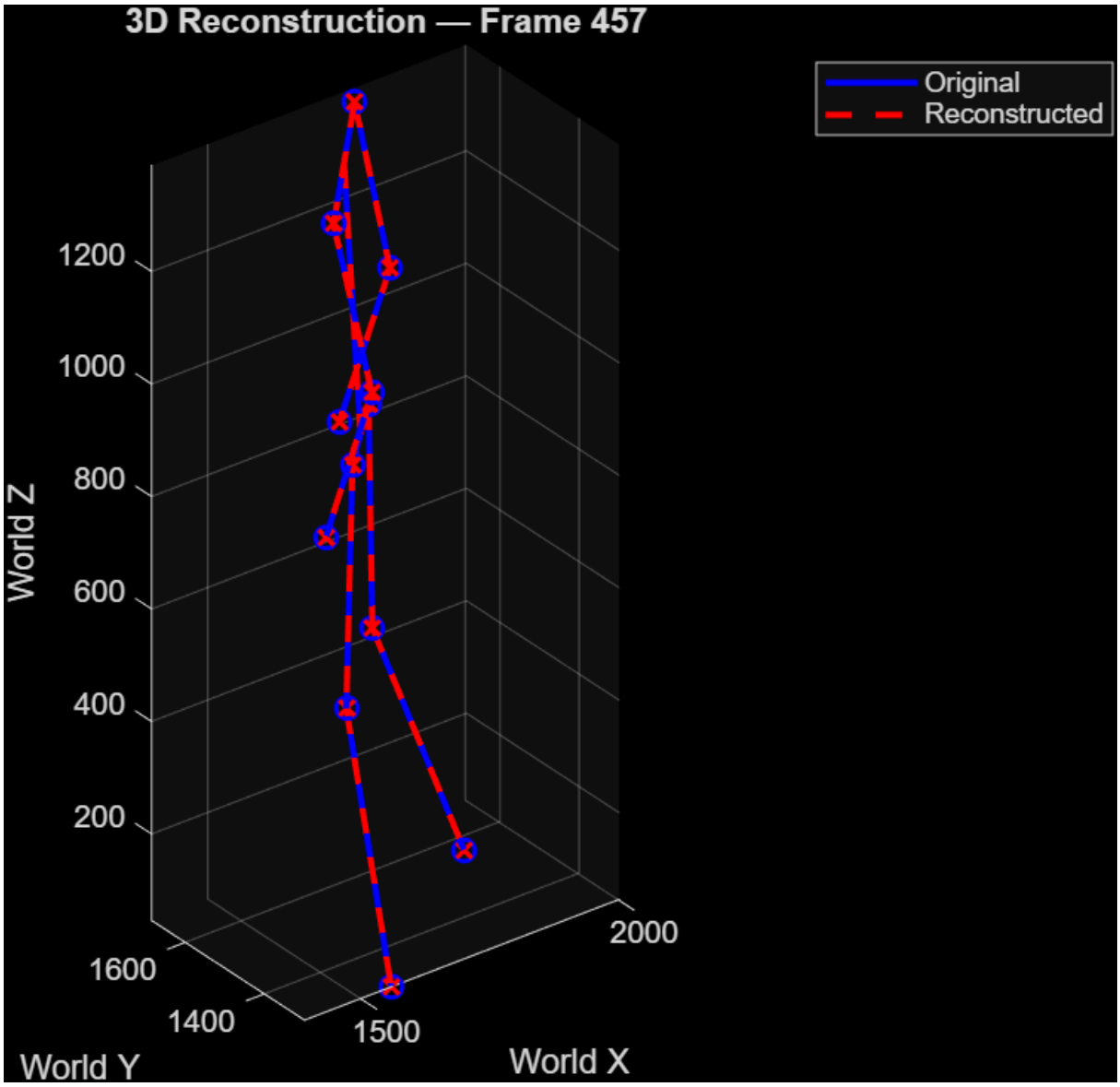



Figure 3. Original and reconstructed 3D skeletons for frame 457. The two skeletons overlap because the reconstruction error is nearly zero.

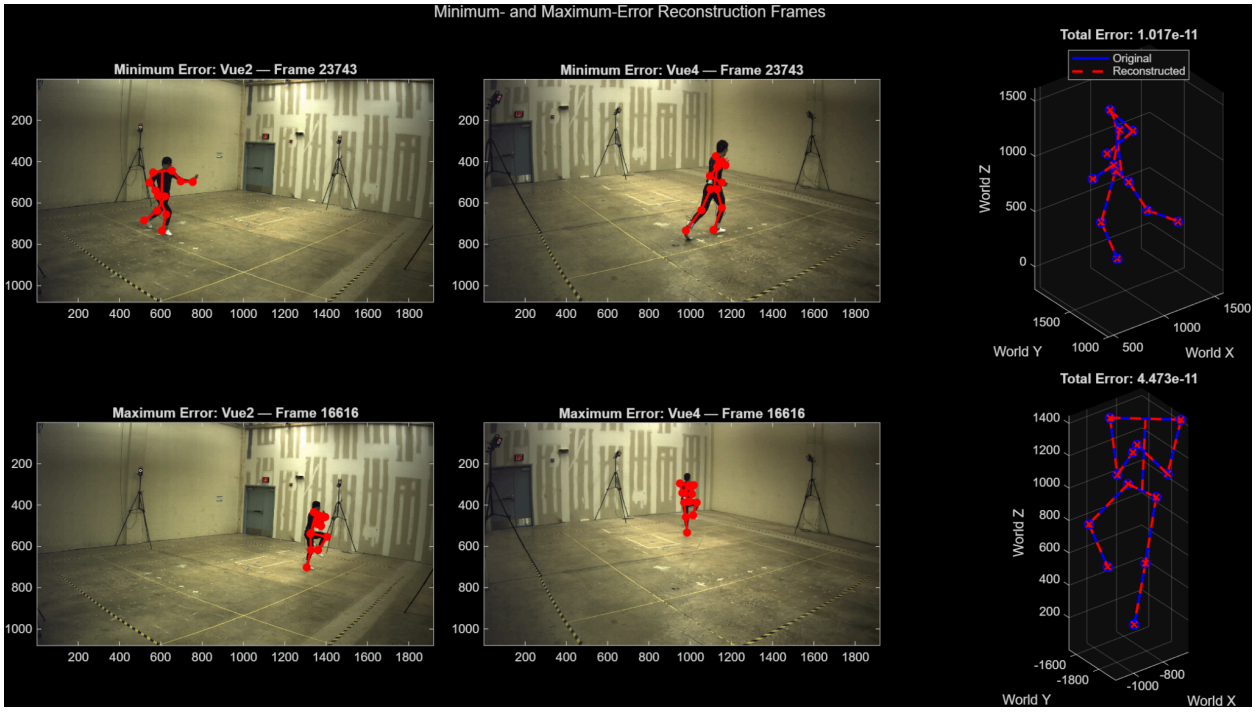



Figure 4. Vue2 projection, Vue4 projection, and 3D reconstruction for the minimum-error frame 23743 and the maximum-error frame 16616.

**Quantitative Results**

Reconstruction accuracy was measured using the Euclidean distance between each original 3D joint (X, Y, Z) and its reconstructed position (X', Y', Z'):

d = sqrt[(X - X')^2 + (Y - Y')^2 + (Z - Z')^2]

The error was computed for all twelve joints in every frame where all confidence values were 1, yielding 25,758 valid frames. For each joint, I computed the mean, standard deviation, minimum, median, and maximum error over the full sequence. I also computed the same five statistics across all joints.

The overall mean reconstruction error was 1.7093 x 10^-12, and the largest individual joint error was 9.5612 x 10^-12. These values are extremely close to zero, indicating that the reconstructed coordinates were effectively identical to the original coordinates. The remaining differences were due to floating-point numerical precision rather than visible reconstruction failure.

The total error for each frame was computed by summing the twelve joint errors in that frame. The minimum total error was 1.0174 x 10^-11 at frame 23743, while the maximum total error was 4.4732 x 10^-11 at frame 16616. Although the plot shows peaks and valleys, the entire sequence remains at a very small numerical scale.

| Joint | Mean | St. Dev. | Minimum | Median | Maximum |
|---|---|---|---|---|---|
| Right Shoulder | 1.67E-12 | 8.36E-13 | 2.27E-13 | 1.53E-12 | 7.34E-12 |
| Right Elbow | 1.68E-12 | 8.41E-13 | 1.14E-13 | 1.54E-12 | 8.22E-12 |
| Right Wrist | 1.67E-12 | 8.47E-13 | 0 | 1.53E-12 | 9.07E-12 |
| Left Shoulder | 1.71E-12 | 8.71E-13 | 1.15E-13 | 1.55E-12 | 8.94E-12 |
| Left Elbow | 1.72E-12 | 8.80E-13 | 0 | 1.56E-12 | 8.95E-12 |
| Left Wrist | 1.72E-12 | 8.85E-13 | 1.15E-13 | 1.57E-12 | 9.56E-12 |
| Right Hip | 1.69E-12 | 8.48E-13 | 0 | 1.55E-12 | 7.25E-12 |
| Right Knee | 1.70E-12 | 8.48E-13 | 0 | 1.55E-12 | 6.90E-12 |
| Right Ankle | 1.73E-12 | 8.63E-13 | 6.39E-14 | 1.59E-12 | 7.67E-12 |
| Left Hip | 1.70E-12 | 8.58E-13 | 1.14E-13 | 1.55E-12 | 7.52E-12 |
| Left Knee | 1.73E-12 | 8.81E-13 | 2.13E-14 | 1.58E-12 | 7.61E-12 |
| Left Ankle | 1.78E-12 | 9.09E-13 | 1.42E-14 | 1.63E-12 | 8.22E-12 |
| Overall | 1.71E-12 | 8.65E-13 | 0 | 1.56E-12 | 9.56E-12 |

Table 2. Euclidean reconstruction-error statistics for each joint and for all joints combined.

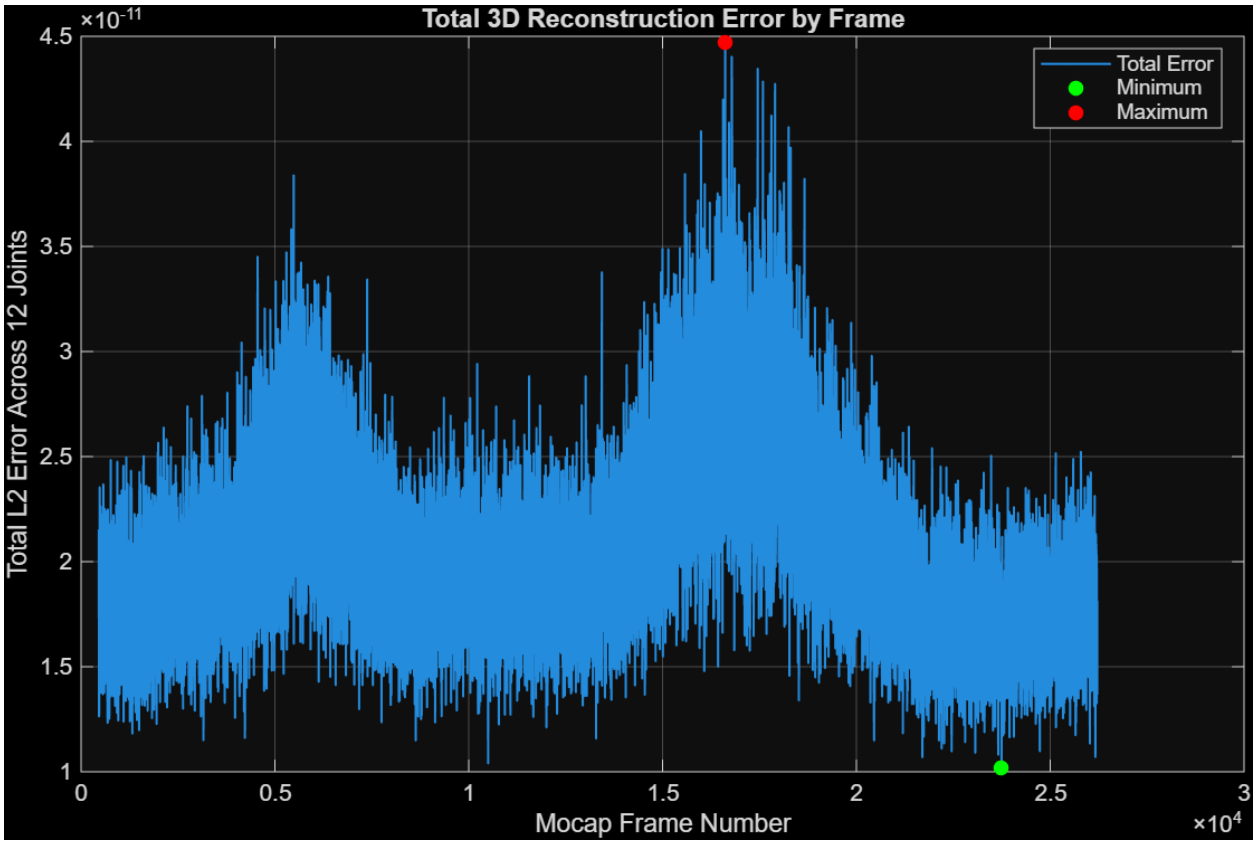



Figure 5. Total Euclidean reconstruction error across the twelve joints for each valid mocap frame. The minimum occurs at frame 23743, and the maximum occurs at frame 16616.

**Epipolar Geometry**

To visualize the epipolar geometry between Vue2 and Vue4, I used the viewing-ray method described in the project instructions. For each projected joint in one camera, I backprojected the pixel into a 3D viewing ray. I selected two points along that ray and projected both points into the other camera. The line through the two projected points formed the corresponding epipolar line.

The same process was repeated in both directions so that epipolar lines were shown in both camera views. Matching joints and their corresponding epipolar lines were displayed using the same colors. The projected joint points lie on, or are extremely close to, their corresponding lines, which shows that the two camera views are geometrically consistent.

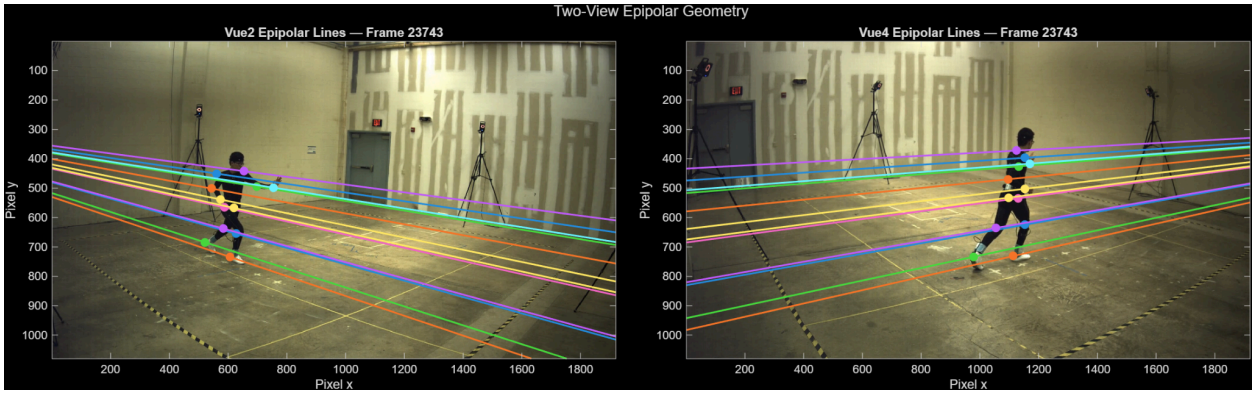
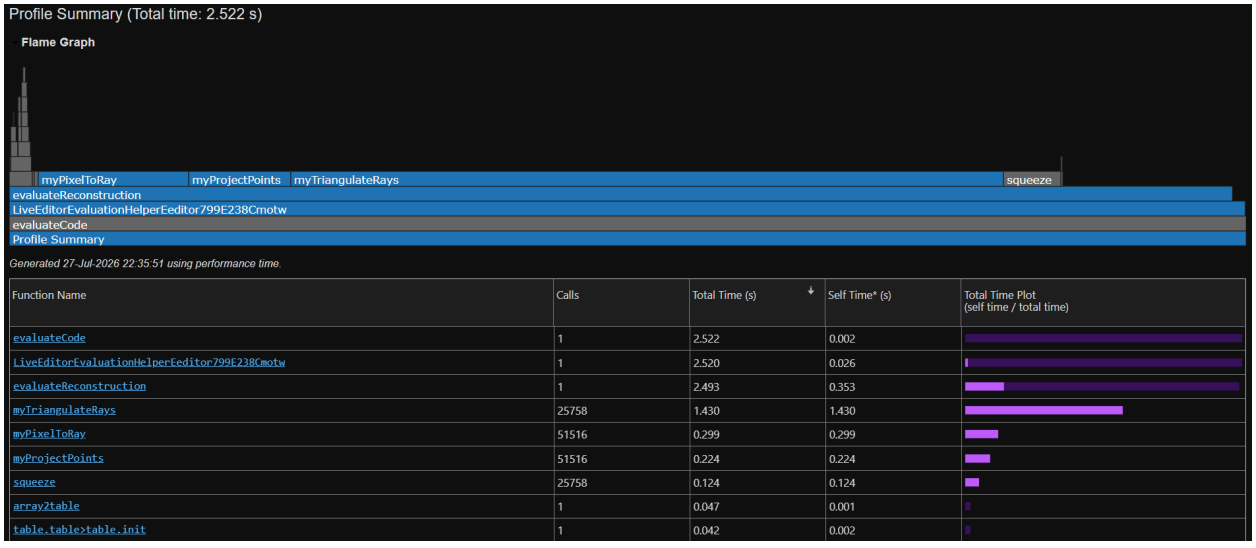



Figure 6. Epipolar lines and corresponding projected joint points in the Vue2 and Vue4 views for frame 23743.

**Algorithm Efficiency**

I used MATLAB's profiler to measure the runtime of the full reconstruction evaluation over all 25,758 valid frames. The complete evaluation took approximately 2.52 seconds. Most of the runtime was spent in 'myTriangulateRays', which took about 1.43 seconds. This was expected because the function solves a small linear system for each pair of viewing rays across all twelve joints and every valid frame.

The inverse-projection function 'myPixelToRay' took about 0.30 seconds, while 'myProjectPoints' took about 0.22 seconds. The remaining time was divided among the evaluation loop and MATLAB's built-in operations. Since the full dataset was processed in only a few seconds, I did not make additional performance changes.




| Function Name | Calls | Total Time (s) | Self Time* (s) | Total Time Plot (self time / total time) |
|---|---|---|---|---|
| evaluateCode | 1 | 2.522 | 0.002 | |
| LiveEditorEvaluationHelperEeditor799E238Cmotw | 1 | 2.520 | 0.026 | |
| evaluateReconstruction | 1 | 2.493 | 0.353 | |
| myTriangulateRays | 25758 | 1.430 | 1.430 | |
| myPixelToRay | 51516 | 0.299 | 0.299 | |
| myProjectPoints | 51516 | 0.224 | 0.224 | |
| squeeze | 25758 | 0.124 | 0.124 | |
| array2table | 1 | 0.047 | 0.001 | |
| table.table>table.init | 1 | 0.042 | 0.002 | |

Figure 7. MATLAB profiler summary for the full reconstruction evaluation. Triangulation consumed the largest share of execution time.

**Conclusion**

This project implemented the complete camera projection and reconstruction pipeline using calibrated camera parameters. The original 3D mocap joints were projected into both camera views, converted back into world-coordinate viewing rays, and reconstructed via two-view triangulation. The projected skeletons followed the subject in both videos, while the reconstructed 3D skeletons overlapped almost exactly with the original skeletons.

Across all 25,758 valid frames, reconstruction errors remained near 10^-12. This indicated that the projection and inverse-projection equations were consistent and that the remaining differences arose from floating-point calculations rather than a noticeable loss of accuracy. The epipolar-line visualization also confirmed the geometric relationship between the two camera views. Overall, the program successfully reconstructed the original 3D joint locations and completed the required quantitative, qualitative, and efficiency evaluations.

**Appendix A. MATLAB Code for Quantitative Evaluation**

```
jointErrors(i, :) = sqrt(sum( ...
           (original3D - reconstructed3D).^2, 2))';
totalError = sum(jointErrors, 2);

% 12 rows, 1 for each joint
statistics = [
   mean(jointErrors, 1)', ...
   std(jointErrors, 0, 1)', ...
   min(jointErrors, [], 1)', ...
   median(jointErrors, 1)', ...
   max(jointErrors, [], 1)'
];

% stats over every joint in every frame
allErrors = jointErrors(:);
overallStatistics = [
   mean(allErrors), ...
   std(allErrors), ...
   min(allErrors), ...
   median(allErrors), ...
   max(allErrors)
];

statistics = [statistics; overallStatistics];
[minimumTotalError, minIndex] = min(totalError);
[maximumTotalError, maxIndex] = max(totalError);
```